

OBJECT ORIENTED PROGRAMMING

CSC166 — Semester II

UNIT 2

Basics of C++ Programming

Lecture Notes by: Dabbal Singh Mahara
Amrit Campus, Thamel, Kathmandu

Reference: Robert Lafore — OOP in C++ (4th Ed.) | Herbert Schildt — C++ The Complete Reference (4th Ed.)

Topics:

Introduction to C++, C++ Program Structure, Character Set & Tokens, Data Types, Type Conversion, Preprocessor Directives, Namespace, I/O Streams and Manipulators, Dynamic Memory Allocation with new and delete, Control Statements, Functions: function overloading, inline functions, default arguments, pass by reference, return by reference, Scope and Storage Class, Pointers: Pointer variables declaration and initialization, Operators in pointers, pointers and Arrays, Pointer and Function

0 Introduction to C++

What is C++?

C++ is a statically typed, compiled, general-purpose programming language created by **Bjarne Stroustrup** in 1979 at Bell Labs as an extension of the C language. Originally called "C with Classes", it was renamed C++ in 1983. The "++" is the increment operator in C, symbolizing that C++ is an improved version of C. It supports multiple programming paradigms — **procedural**, **object-oriented**, and **generic** — making it one of the most versatile languages in software development. C++ compiles directly to native machine code, making it extremely fast and suitable for performance-critical applications.

Key Features of C++

- **Low-level Memory Control:** Direct memory management using pointers, new, and delete operators.
- **Object-Oriented Programming (OOP):** Supports classes, inheritance, polymorphism, and encapsulation.
- **Templates (Generic Programming):** Write reusable code that works with any data type at compile time.
- **Standard Template Library (STL):** A rich library of containers (vector, map), algorithms, and iterators.
- **C Compatibility:** C++ is backward compatible with C; most C programs compile as valid C++.
- **High Performance:** Compiled to native machine code; no runtime interpreter overhead.

Where is C++ Used?

C++ is widely used across many domains: **Game engines** (Unreal Engine), **operating systems** (Windows kernel components), **browsers** (Google Chrome), **embedded systems**, **financial systems**, and **AI/robotics**.

Brief History

1979	Bjarne Stroustrup begins developing “C with Classes” at Bell Labs
1983	Renamed to “C++”; virtual functions, operator overloading added
1998	ISO C++98 — first official international standard published
2011	C++11 (Modern C++) — auto, lambdas, smart pointers, range-based for
2023	C++23 — latest standard with improved modules and ranges library

1. C++ Program Structure

Every C++ program has a standard structure that the compiler expects. Understanding this structure is the first step to writing correct C++ programs.

```
// Example: A minimal C++ program
#include <iostream>      // Preprocessor directive
using namespace std;    // Namespace declaration

int main() {            // Entry point
    cout << "Hello, World!" << endl;
    return 0;           // Return 0 = success
}
```

Key Components:

- **Comments:** Lines beginning with // (single-line) or enclosed in /* ... */ (multi-line) are ignored by the compiler. Used for documentation.
- **Preprocessor Directives:** Lines beginning with # are processed before compilation. #include <iostream> inserts the iostream header file.
- **Namespace Declaration:** using namespace std; allows use of standard library names (cout, cin, endl) without the std:: prefix.
- **main() Function:** Execution begins here. Every C++ program must have exactly one main() function. It returns an int (0 = success, non-zero = error).
- **Statements:** Instructions ended with a semicolon (;). The compiler does not care about whitespace or line breaks.
- **Blocks:** Groups of statements enclosed in curly braces { } form a block (compound statement).

 **Note:** In C++, the order of sections is: Preprocessor Directives → Global Declarations → Function Definitions → main(). The compiler processes the file top to bottom.

2. Character Set and Tokens

2.1 Character Set

The character set is the set of valid characters that can be used in a C++ program. C++ uses the ASCII character set, which includes:

- Uppercase letters: A–Z
- Lowercase letters: a–z
- Digits: 0–9
- Special symbols: + - * / % = & | ! ~ ^ < > () { } [] ; : ' " , . ? # _ \
- Whitespace: space, tab, newline, carriage return

2.2 Tokens

A token is the smallest meaningful unit in a C++ program. The compiler breaks source code into tokens during the first phase of compilation (lexical analysis). C++ has six types of tokens:

Token Type	Description & Examples
Keywords	Reserved words with predefined meaning. Cannot be used as identifiers. Examples: int, float, if, else, while, for, return, class, public, private, new, delete, void
Identifiers	Names given by the programmer to variables, functions, classes, etc. Rules: start with a letter or underscore, contain letters/digits/underscores, case-sensitive, cannot be a keyword. E.g.: myVariable, _count, BankAccount
Literals	Fixed values written directly in code. Types: integer (42), floating-point (3.14), character ('A'), string ("Hello"), boolean (true/false)
Operators	Symbols that perform operations. Arithmetic: + - * / %. Relational: == != < > <= >= . Logical: && !. Assignment: = += -= *=. Bitwise: & ^ ~ << >>
Punctuators	Symbols used for structure/syntax. Examples: ; (statement end), , (separator), { } (block), [] (array), () (function), :: (scope resolution), -> (pointer member access)
String Literals	Sequence of characters in double quotes. E.g.: "Hello, World!". The compiler appends a null terminator \0 at the end automatically.

 **Note:** Identifiers in C++ are case-sensitive: myVar, MyVar, and MYVAR are three different identifiers. Keywords like int, float, and class are always lowercase.

3. Data Types

A data type specifies the kind of data a variable can hold and the operations that can be performed on it. C++ provides a rich set of data types.

3.1 Fundamental (Primitive) Data Types

Data Type	Size (typical)	Range / Description
int	4 bytes	-2,147,483,648 to 2,147,483,647 (whole numbers)
short int	2 bytes	-32,768 to 32,767
long int	4–8 bytes	Larger range of whole numbers
unsigned int	4 bytes	0 to 4,294,967,295 (no negatives)
float	4 bytes	$\pm 3.4 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ (6–7 decimal digits precision)
double	8 bytes	$\pm 1.7 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$ (15–16 decimal digits precision)
long double	10–16 bytes	Extended precision floating-point
char	1 byte	Single character or small integer (-128 to 127 or 0 to 255)
bool	1 byte	true (1) or false (0)
void	0 bytes	Represents the absence of a type. Used for functions that return nothing.

3.2 Derived Data Types

- Arrays: Collection of elements of the same type. E.g.: `int arr[10];`
- Pointers: Variables that store memory addresses. E.g.: `int* ptr;`
- References: Alias for another variable. E.g.: `int& ref = x;`
- Functions: A derived type that maps input types to an output type.

3.3 User-Defined Data Types

- struct: A collection of variables of different types under one name.
- class: Like struct but with methods and access control. The core of OOP.
- union: Members share the same memory location.
- enum: A set of named integer constants.

3.4 Type Conversion

Type conversion (or type casting) is the process of converting a value from one data type to another.

a) Implicit Conversion (Automatic / Widening)

The compiler automatically converts a smaller type to a larger type without data loss. The hierarchy: char → short → int → long → float → double → long double.

```
int i = 10;
double d = i; // int automatically converted to double → d = 10.0
float f = 3.14;
double g = f; // float automatically converted to double
```

b) Explicit Conversion (Casting / Narrowing)

The programmer manually forces a conversion using a cast operator. This may cause data loss.

```
double pi = 3.14159;
int x = (int) pi; // C-style cast → x = 3 (decimal part lost)
int y = static_cast<int>(pi); // C++ style cast → y = 3

// C++ cast operators:
// static_cast<T>(expr) — compile-time type conversion (most common)
// dynamic_cast<T>(expr) — safe downcasting for polymorphic types
// reinterpret_cast<T>(expr) — bit-level reinterpretation (use with care)
// const_cast<T>(expr) — add/remove const qualifier
```

 **Note:** Implicit conversion from double to int is a narrowing conversion and may cause warnings. Always use explicit casts when narrowing to make intent clear.

4. Preprocessor Directives

Preprocessor directives are instructions to the C++ preprocessor, which runs before the actual compilation. They begin with # and do not end with a semicolon. They are processed textually — they do not generate machine code directly.

4.1 #include

Inserts the contents of another file into the current file. Used to include header files that declare standard library functions and classes.

```
#include <iostream> // Standard header (angle brackets = system header)
#include <string>
#include <cmath>
#include "myfile.h" // User-defined header (quotes = local file)
```

4.2 #define (Macro Definition)

Creates a macro — a name that the preprocessor replaces with a given text before compilation.

```
#define PI 3.14159      // Object-like macro
#define MAX(a,b) ((a)>(b)?(a):(b)) // Function-like macro

// Usage:
double area = PI * r * r; // Preprocessor replaces PI with 3.14159
int m = MAX(x, y);        // Replaced with ((x)>(y)?(x):(y))
```



Note: Prefer const variables and inline functions over #define macros in C++. Macros have no type checking and can cause subtle bugs.

4.3 Conditional Compilation (#ifdef, #ifndef, #endif)

Allows selective compilation of code blocks based on whether a macro is defined. Commonly used for include guards and platform-specific code.

```
// Include guard — prevents double inclusion of a header file
#ifndef MYHEADER_H
#define MYHEADER_H

// ... header contents ...

#endif // MYHEADER_H
```

4.4 Other Directives

- `#undef MACRO_NAME` — undefines a macro.
- `#pragma` — compiler-specific instructions (e.g., `#pragma once` as an alternative include guard).
- `#error "message"` — forces a compilation error with a custom message.

5. Namespace

A namespace is a declarative region that provides a scope for identifiers (variables, functions, classes). Namespaces prevent name collisions in large projects where different modules or libraries may define the same names.

5.1 Defining a Namespace

```
namespace Physics {  
    double PI = 3.14159;  
    double gravity = 9.8;  
    void display() { cout << "Physics namespace"; }  
}  
  
namespace Math {  
    double PI = 3.14159265; // Same name, different namespace — no conflict!  
}
```

5.2 Accessing Namespace Members

```
// Method 1: Scope Resolution Operator (::)  
cout << Physics::PI;    // Access PI from Physics namespace  
cout << Math::PI;      // Access PI from Math namespace  
  
// Method 2: using declaration (imports one name)  
using Physics::gravity;  
cout << gravity;        // No prefix needed  
  
// Method 3: using directive (imports all names from namespace)  
using namespace std;    // Most common — imports cout, cin, endl, etc.  
cout << "Hello";        // No std:: prefix needed
```

 **Note:** using namespace std; is convenient but in large projects it can cause name conflicts. In professional code, prefer the std:: prefix or specific using declarations.

5.3 The std Namespace

All standard C++ library entities (cout, cin, string, vector, endl, etc.) are defined inside the std namespace. That is why we write std::cout or use 'using namespace std;' at the top of our programs.

6. Input/Output Streams and Manipulators

C++ uses a stream-based I/O system. A stream is a sequence of bytes flowing between the program and a device (keyboard, screen, file). Streams are defined in the <iostream> header.

6.1 Standard Stream Objects

Stream Object	Direction	Description
Cin	Input	Standard input stream. Connected to the keyboard by default.
Cout	Output	Standard output stream. Connected to the screen by default.
Cerr	Output	Standard error stream. Unbuffered. For error messages.
Clog	Output	Standard log stream. Buffered. For log messages.

6.2 Insertion and Extraction Operators

```
#include <iostream>
using namespace std;

int main() {
    int age;
    string name;

    cout << "Enter your name: "; // << is insertion operator (output)
    cin >> name;                // >> is extraction operator (input)
    cout << "Enter your age: ";
    cin >> age;
    cout << "Hello, " << name << "! You are " << age << " years old." << endl;
    return 0;
}
```

 **Note:** cin >> skips whitespace and stops at the next whitespace. To read a full line including spaces, use: getline(cin, variableName);

6.3 Output Manipulators

Manipulators are special functions that modify the behavior of streams. Include <iomanip> for most manipulators.

Manipulator	Effect & Example
Endl	Inserts a newline and flushes the buffer. cout << "Hi" << endl;
"\n"	Inserts a newline only (no flush). Faster than endl. cout << "Hi\n";
setw(n)	Sets field width for the next output item. cout << setw(10) << x;

setprecision(n)	Sets decimal places for floating-point. <code>cout << setprecision(2) << 3.14159;</code>
Fixed	Displays float in fixed-point notation. <code>cout << fixed << setprecision(2) << 3.14;</code>
scientific	Displays float in scientific notation. <code>cout << scientific << 3.14;</code>
left / right	Aligns output left or right within field width.
setfill(c)	Sets fill character for padding. <code>cout << setfill('0') << setw(5) << 42; → 00042</code>
hex / oct / dec	Outputs integers in hexadecimal, octal, or decimal.
boolalpha	Outputs bool as 'true'/'false' instead of 1/0.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double pi = 3.14159;
    cout << fixed << setprecision(2) << pi << endl; // Output: 3.14
    cout << setw(10) << "Name" << setw(5) << "Age" << endl;
    cout << setfill('-') << setw(15) << "" << endl; // Output: -----
    cout << hex << 255 << endl; // Output: ff
}
```

7. Dynamic Memory Allocation — new and delete

In C++, memory can be allocated in two ways: statically (at compile time, on the stack) or dynamically (at runtime, on the heap). Dynamic allocation is done using the new and delete operators, which replace C's malloc()/free().

7.1 The new Operator

new allocates memory on the heap at runtime and returns a pointer to the allocated memory.

```
// Syntax: pointer = new DataType;
int* p = new int; // Allocates memory for one int
*p = 42; // Store value at that address

// Allocate and initialize in one step:
int* q = new int(100); // Allocates int, initializes to 100

// Allocate an array:
int* arr = new int[10]; // Allocates array of 10 ints on heap
arr[0] = 5; // Access like a normal array
```

7.2 The delete Operator

delete releases dynamically allocated memory back to the system. Failing to delete causes memory leaks — memory that is allocated but never freed.

```
delete p;      // Free a single object
delete q;
delete[] arr;   // Free a dynamically allocated array (use delete[], not delete)

// After deleting, set pointer to nullptr to avoid dangling pointers:
p = nullptr;
arr = nullptr;
```

new / delete vs malloc / free

Feature	new / delete (C++)	malloc / free (C)
Type safety	Yes — returns typed pointer	No — returns void*
Constructor/Destructor	Called automatically	Not called
Failure behavior	Throws std::bad_alloc exception	Returns NULL
Syntax	int* p = new int(5);	int* p = (int*)malloc(sizeof(int));

 **Note:** Always pair every new with a delete, and every new[] with a delete[]. Mismatching (using delete on a new[] allocation) causes undefined behavior.

8. Control Statements

Control statements alter the sequential flow of execution. C++ supports selection, iteration, and jump statements.

8.1 Selection Statements

a) if / if-else / else-if Ladder

```
int marks = 75;
if (marks >= 80)    cout << "Distinction";
else if (marks >= 60) cout << "First Division";
else if (marks >= 45) cout << "Second Division";
else               cout << "Fail";
```

b) switch Statement

Efficient multi-way branch based on an integer or enum expression.

```
int day = 3;
switch (day) {
    case 1: cout << "Monday"; break;
    case 2: cout << "Tuesday"; break;
    case 3: cout << "Wednesday"; break;
    default: cout << "Other day";
}
```

 **Note:** Without break, execution falls through to the next case. This is a common bug. Always include break unless fall-through is intentional.

8.2 Iteration Statements (Loops)

a) for Loop — when number of iterations is known

```
for (int i = 0; i < 5; i++) {
    cout << i << " "; // Output: 0 1 2 3 4
}
```

b) while Loop — when condition is checked before iteration

```
int i = 0;
while (i < 5) {
    cout << i << " ";
    i++;
}
```

c) do-while Loop — body executes at least once

```
int i = 0;
do {
    cout << i << " ";
    i++;
} while (i < 5);
```

8.3 Jump Statements

- **break:** Exits the nearest enclosing loop or switch immediately.
- **continue:** Skips the rest of the current loop iteration and jumps to the next.
- **return:** Exits the current function and optionally returns a value to the caller.
- **goto:** Unconditionally jumps to a labeled statement. Strongly discouraged in C++ (use structured control flow instead).

9. Functions

A function is a named block of code that performs a specific task. Functions promote modularity, reusability, and readability. In C++, functions must be declared before use.

9.1 Function Structure

```
// Function Declaration (Prototype) — tells compiler about the function
returnType functionName(parameterList);

// Function Definition — the actual implementation
returnType functionName(parameterList) {
    // body
    return value; // must match returnType
}

// Example:
int add(int a, int b);    // declaration

int add(int a, int b) {    // definition
    return a + b;
}

int main() {
    int result = add(3, 4); // function call → result = 7
}
```

9.2 Function Overloading

C++ allows multiple functions with the same name but different parameter lists (different number, type, or order of parameters). The compiler selects the correct version based on the arguments at the call site. This is compile-time polymorphism.

```
int add(int a, int b)    { return a + b; }
double add(double a, double b) { return a + b; }
```

```
int add(int a, int b, int c) { return a + b + c; }

cout << add(2, 3);      // calls int version → 5
cout << add(2.5, 3.5);  // calls double version → 6.0
cout << add(1, 2, 3);   // calls three-argument version → 6
```

 **Note:** Function overloading is resolved by the function signature (name + parameter types). The return type alone cannot distinguish overloaded functions.

9.3 Inline Functions

An inline function is a request to the compiler to insert the function's body directly at the call site instead of generating a function call. This eliminates function call overhead for small, frequently-called functions.

```
inline int square(int x) { return x * x; }

// When you write: int s = square(5);
// Compiler treats it as: int s = 5 * 5;
// No function call overhead!
```

 **Note:** inline is a hint, not a guarantee. The compiler may ignore the inline request for complex functions. Modern compilers often inline automatically without the keyword.

9.4 Default Arguments

C++ allows function parameters to have default values. If the caller does not provide an argument, the default is used. Default arguments must be specified from right to left (trailing parameters first).

```
void greet(string name, string greeting = "Hello") {
    cout << greeting << ", " << name << "!" << endl;
}

greet("Alice");           // Output: Hello, Alice!
greet("Bob", "Welcome");  // Output: Welcome, Bob!
```

9.5 Pass by Value vs Pass by Reference

a) Pass by Value (default)

A copy of the argument is passed. Changes inside the function do not affect the original variable.

```
void increment(int x) { x++; } // x is a copy
int a = 5;
increment(a);
cout << a; // Output: 5 (unchanged!)
```

b) Pass by Reference

The function receives a reference (alias) to the actual variable. Changes inside the function DO affect the original variable. Use & in the parameter declaration.

```
void increment(int& x) { x++; } // x is a reference to original
int a = 5;
increment(a);
cout << a; // Output: 6 (changed!)

// Swap using pass by reference:
void swap(int& a, int& b) {
    int temp = a; a = b; b = temp;
}
```

9.6 Return by Reference

A function can return a reference to a variable, allowing the function call to be used as an lvalue (left side of assignment). Use with care — never return a reference to a local variable (it will be destroyed when the function exits).

```
int& getElement(int arr[], int index) {
    return arr[index]; // Returns reference to array element
}

int main() {
    int arr[] = {10, 20, 30};
    getElement(arr, 1) = 99; // Modifies arr[1] directly
    cout << arr[1]; // Output: 99
}
```

9.7 Scope and Storage Class

Scope defines where a variable is visible (accessible). Storage class defines its lifetime and storage location.

Scope Types:

- **Local Scope:** Variable declared inside a function or block. Visible only within that block.
- **Global Scope:** Variable declared outside all functions. Visible throughout the entire file.

- **Block Scope:** Variable visible only within the specific { } block.
- **Function Prototype Scope:** Parameter names in a prototype have this scope (basically ignored).

Storage Classes:

Storage Class	Keyword	Lifetime	Default Value	Description
Automatic	auto	Function duration	Garbage	Default for local variables. Created on stack when function is called, destroyed when function exits.
Register	register	Function duration	Garbage	Suggests storing in CPU register for speed. Modern compilers optimize automatically — rarely used.
Static	static	Program duration	0	Variable persists between function calls. Declared inside a function but keeps its value. Global static limits scope to the file.
External	extern	Program duration	0	Declares a variable defined in another file. Used for global variables shared across multiple source files.

```
void counter() {
    static int count = 0; // Initialized only once; persists between calls
    count++;
    cout << "Call #" << count << endl;
}
// counter() → Call #1
// counter() → Call #2
// counter() → Call #3 (count is NOT reset to 0 each call)
```

10. Pointers

A pointer is a variable that stores the memory address of another variable. Pointers are one of C++'s most powerful (and most misused) features. Understanding pointers is essential for dynamic memory, arrays, and data structures.

10.1 Pointer Declaration and Initialization

```
// Declaration: DataType* pointerName;
int* p;    // p is a pointer to int (uninitialized — DANGEROUS)
double* dp; // dp is a pointer to double
```

```
// Initialization:
int x = 42;
int* p = &x; // p holds the address of x (& = address-of operator)

cout << x; // Output: 42 (value of x)
cout << &x; // Output: 0x7ffee4c (address of x — varies)
cout << p; // Output: 0x7ffee4c (same address, stored in p)
cout << *p; // Output: 42 (* = dereference operator — gets value AT address)
```

 **Note:** Always initialize pointers before use. An uninitialized pointer contains a garbage address. Dereferencing it causes undefined behavior (often a crash). Use `int* p = nullptr`; if no address is available yet.

10.2 Pointer Operators

Operator	Name	Description & Example
&	Address-of	Returns the memory address of a variable. <code>int* p = &x;</code>
*	Dereference	Accesses the value at the address stored in a pointer. <code>cout << *p;</code>
->	Arrow	Accesses member of a struct/class through a pointer. <code>ptr->member</code>
[]	Subscript	Array-style access through a pointer. <code>*(p+i)</code> is same as <code>p[i]</code>

10.3 Pointer Arithmetic

Arithmetic operations on pointers are valid. Adding 1 to a pointer moves it forward by the size of the pointed-to type (not by 1 byte).

```
int arr[] = {10, 20, 30, 40, 50};
int* p = arr; // p points to arr[0]

cout << *p; // 10 (arr[0])
p++; // p now points to arr[1] (moved 4 bytes forward on 32-bit int)
cout << *p; // 20 (arr[1])
p += 2; // p now points to arr[3]
cout << *p; // 40 (arr[3])

// Difference between pointers:
int* q = &arr[4];
cout << (q - p); // 1 (one int element apart)
```


10.4 Pointers and Arrays

An array name is a constant pointer to its first element. This means arrays and pointers are closely related in C++.

```
int arr[] = {10, 20, 30};
int* p = arr;    // arr decays to &arr[0]

// These are all equivalent:
cout << arr[1];  // Standard array access
cout << *(arr + 1); // Pointer arithmetic
cout << p[1];    // Pointer used as array
cout << *(p + 1); // Dereference pointer arithmetic
```

10.5 Pointers and Functions

a) Passing Pointer to Function

Passing a pointer allows the function to modify the original variable (like pass by reference).

```
void doubleValue(int* p) {
    *p = (*p) * 2; // Modify the value at the address
}
int x = 5;
doubleValue(&x); // Pass address of x
cout << x;      // Output: 10
```

b) Returning Pointer from Function

```
int* createArray(int size) {
    int* arr = new int[size]; // Allocate on heap
    for (int i = 0; i < size; i++) arr[i] = i * i;
    return arr;               // Return pointer — safe (heap persists)
}
int* myArr = createArray(5);
// Use myArr...
delete[] myArr;              // Must free manually!
```

 **Note:** Never return a pointer to a local (stack) variable. Local variables are destroyed when the function returns, leaving a dangling pointer. Only return pointers to heap-allocated memory or to variables with a longer lifetime.

c) nullptr

nullptr (C++11 onwards) is the null pointer constant. A null pointer does not point to any valid memory location. Always check for nullptr before dereferencing.

```
int* p = nullptr; // p points to nothing
if (p != nullptr) {
    cout << *p;    // Safe — only dereference if not null
}
```

11. Unit Summary

Unit 2 covers the building blocks of C++ programming. Here is a concise overview:

Topic	Key Points
Program Structure	#include, namespace, main(), statements, blocks, return 0
Character Set & Tokens	Keywords, identifiers, literals, operators, punctuators
Data Types	int, float, double, char, bool, void, arrays, pointers, user-defined types
Type Conversion	Implicit (widening) vs Explicit (casting); static_cast<T>()
Preprocessor Directives	#include, #define, #ifdef/#ifndef/#endif, #pragma
Namespace	Scope for identifiers; std namespace; :: operator; using
I/O Streams	cin, cout, cerr; << and >>; manipulators: setw, setprecision, fixed, hex
Dynamic Memory	new (allocate), delete (free); new[] and delete[] for arrays; avoid leaks
Control Statements	if/else, switch/break, for, while, do-while, break, continue, return
Function Overloading	Same name, different signature; compile-time polymorphism
Inline Functions	inline keyword; body inserted at call site; reduces overhead
Default Arguments	Right-to-left defaults; caller can omit trailing arguments
Pass by Reference	& in parameter; modifies original; used for swap, output parameters
Return by Reference	Returns alias; never return reference to local variable
Scope & Storage Class	auto, register, static (persists), extern (shared across files)
Pointer Basics	Stores address (&); dereferences (*); nullptr for safety
Pointer Arithmetic	p++ moves by sizeof(type); arrays and pointers are interchangeable
Pointers & Functions	Pass pointer to modify original; return heap pointer (never stack)

12. Review Questions

1. Describe the structure of a C++ program. What is the role of each section?
2. What are tokens in C++? List and explain all six types with examples.
3. Explain the difference between implicit and explicit type conversion. Give code examples of each.
4. What is a preprocessor directive? Explain `#include`, `#define`, and `#ifndef` with examples.
5. What is a namespace? Why is it needed? How do you define and access a namespace?
6. Differentiate between `endl` and `'\n'`. When would you prefer one over the other?
7. Explain the difference between `new/delete` and `malloc/free`. Why does C++ prefer `new/delete`?
8. What is the difference between a while loop and a do-while loop? Give an example where do-while is more appropriate.
9. What is function overloading? How does the compiler resolve which function to call? Can two functions differ only in return type?
10. Explain inline functions. When should you use them and what are their limitations?
11. Differentiate between pass by value and pass by reference. Write a swap function using both approaches.
12. What is a storage class? Explain the behavior of static local variables with an example.
13. What is a pointer? Explain pointer declaration, initialization, and dereferencing with an example.
14. What is pointer arithmetic? If `int* p = arr`, what does `p+2` point to?
15. Write a function that takes an array and its size as arguments (using a pointer) and returns the maximum element.